

Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Лабораторна робота №2

з дисципліни «Об'єктно-орієнтоване програмування»

на тему:

**«Розробка графічного редактора об'єктів на базі об'єктно-орієнтованого
підходу»**

Виконав:
студент групи ІМ-55
Присяжний С. О.
Номер у списку: 25

Перевірив:
Рекечинський Д. О.

Київ 2026

Мета роботи:

Отримати стійкі практичні вміння та навички застосування фундаментальних принципів об'єктно-орієнтованого програмування: інкапсуляції, абстракції типів даних, успадкування та динамічного поліморфізму. Спроекувати та практично реалізувати модульний графічний редактор двовимірних геометричних об'єктів з нативним системним меню, підтримкою інтерактивного введення маніпулятором миша з динамічним відображенням «гумового» сліду та фіксованим буфером збереження графічних примітивів.

Завдання:

1. Спроекувати об'єктно-орієнтовану структуру графічного редактора на основі базових абстрактних класів Shape (графічні примітиви) та Editor (контролери введення).
2. Реалізувати ієрархію класів геометричних фігур (PointShape, LineShape, RectShape, EllipseShape), що інкапсулюють власні геометричні координати та перевизначають чисто віртуальний метод відображення show().
3. Створити ієрархію класів редакторів введення фігур (PointEditor, LineEditor, RectEditor, EllipseEditor), похідних від базового класу ShapeEditor, з підтримкою обробки подій миші (OnLButtonDown, OnMouseMove, OnLButtonUp) та малювання динамічного «гумового» сліду OnPaint().
4. Реалізувати клас-координатор ShapeObjectsEditor, що керує статичним масивом показників pshape заданого розміру N, інкапсулює перемикання поточного редактора (currentEditor) та забезпечує повне делегування віконних подій.
5. Створити головне вікно графічного застосунку із системним меню («Файл», «Об'єкти», «Довідка»), нижньою панеллю стану (Status Bar) та динамічною індикацією поточного режиму у заголовку вікна згідно індивідуального варіанта.
6. Скомпілювати, протестувати та налагодити створену систему, перевірити коректність поліморфного відображення об'єктів, підготувати схеми/діаграми та оформити звіт.

Розрахунок та характеристика індивідуального варіанта:

Номер індивідуального варіанта визначається номером студента в академічному журналі групи: $Ж = 25$.

Відповідно до формул, наведених у методичних вказівках до лабораторної роботи №2, отримано такі розрахункові параметри:

1. Організація масиву об'єктів Shape:

Умова вибору: якщо $(Ж \bmod 3 = 0)$ — динамічний масив, інакше — статичний масив.
Обсяг масиву $N = Ж + 100$.

Розрахунок: $25 \bmod 3 = 1 \neq 0 \Rightarrow$ обирається статичний масив показників на базовий клас Shape.

Кількість елементів: $N = 25 + 100 = 125$ елементів. У коді проєкту масив реалізовано як фіксований масив посилань `pcshare` довжиною 125 елементів з інкапсульованим лічильником `shapeCount`.

2. «Гумовий» слід при введенні об'єктів:

Умова вибору стилю та кольору: $(Ж \bmod 4)$:

0 — суцільна лінія чорного кольору; 1 — суцільна лінія червоного кольору; 2 — суцільна лінія синього кольору; 3 — пунктирна чорна лінія.

Розрахунок: $25 \bmod 4 = 1 \Rightarrow$ суцільна лінія червоного кольору (Solid Red Line, колір #ef4444). Червоний контур динамічно відмальовується від точки натискання до поточного положення курсору миші.

3. Різновид введення та відображення прямокутника (Rect):

- Спосіб введення: $(Ж \bmod 2)$:

0 — по двох протилежних кутах; 1 — від центру до одного з кутів.

Розрахунок: $25 \bmod 2 = 1 \Rightarrow$ введення прямокутника від центру до одного з кутів (точка кліку задає центр прямокутника, а рух миші формує симетричне розширення контуру $dx = |xend - xstart|$, $dy = |yend - ystart|$).

- Спосіб відображення: $(Ж \bmod 5)$:

0 — чорний контур з білим заповненням; 1 або 2 — чорний контур з кольоровим заповненням; 3 або 4 — чорний контур без заповнення.

Розрахунок: $25 \bmod 5 = 0 \Rightarrow$ чорний контур з білим заповненням (Black Outline, White Fill #ffffff), що непрозоро перекриває фігури під ним.

4. Різновид введення та відображення еліпса (Ellipse):

- Спосіб введення: $(Ж \bmod 2)$:

0 — від центру до одного з кутів; 1 — по двох протилежних кутах охоплюючого прямокутника.

Розрахунок: $25 \bmod 2 = 1 \Rightarrow$ введення еліпса по двох протилежних кутах охоплюючого прямокутника.

- Спосіб відображення: $(Ж \bmod 5)$:

1 — чорний контур з білим заповненням; 3 або 4 — з кольоровим заповненням; 0 або 2 — без заповнення.

Розрахунок: $25 \bmod 5 = 0 \Rightarrow$ чорний контур еліпса без заповнення (прозоре внутрішнє поле, крізь яке видно раніше намальовані елементи).

5. Позначка поточного типу об'єкта, що вводиться:

Умова вибору: $(Ж \bmod 2)$:

0 — у підпунктах системного меню (`OnInitMenuPopup`); 1 — у заголовку вікна програми.

Розрахунок: $25 \bmod 2 = 1 \Rightarrow$ виведення поточної геометричної форми в заголовку вікна програми (текстовий статус «Графічний редактор — [Режим: <Форма>] (Об'єктів: К/125)», що передається з процесу `Renderer` у `Main Process` через IPC-міст та додатково відображається у рядку стану).

Схеми та діаграми структури програми:

Згідно з методичними вимогами розроблено UML-діаграму класів (Class Diagram) та діаграму архітектури модулів і взаємодії процесів проєкту. Програма побудована на чіткому розділенні відповідальностей згідно з принципом єдиного обов'язку (Single Responsibility Principle) та патерном проектування «Стан/Стратегія» (State/Strategy) для заміни поведінки інструментів редагування.

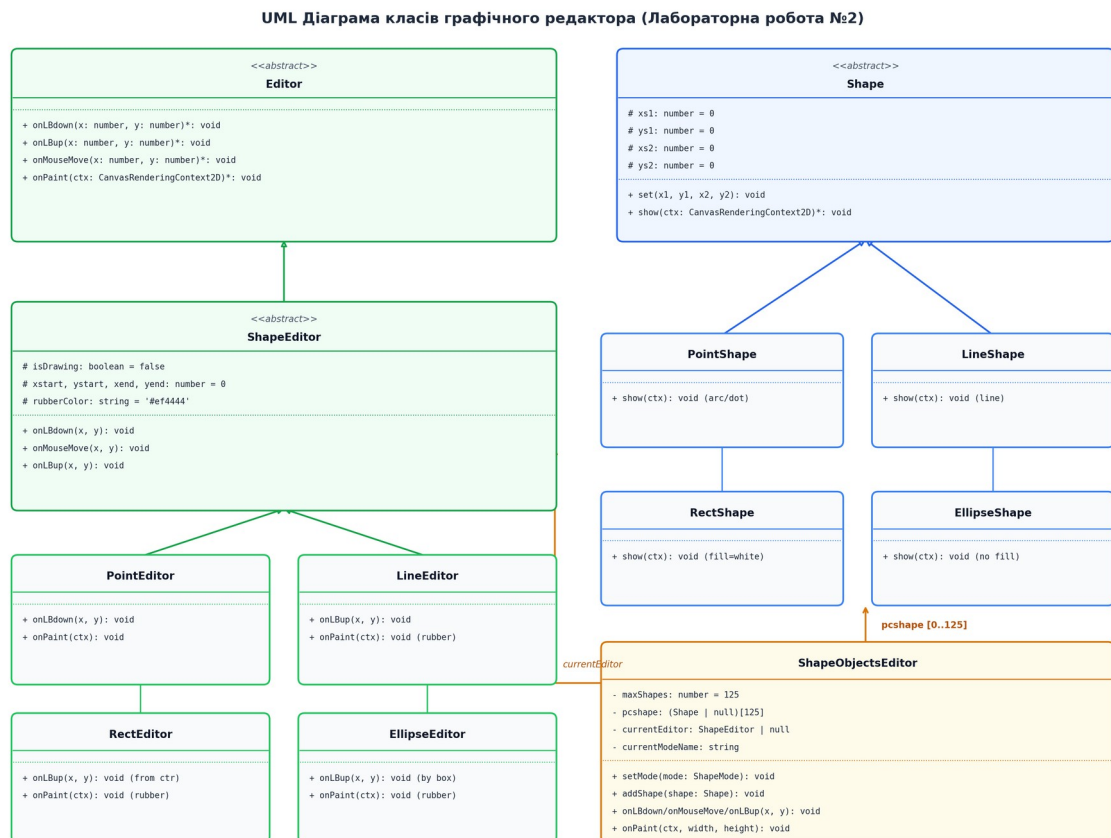


Рис. 1. UML діаграма класів графічного редактора об'єктів (ієрархії Shape, Editor та клас-координатор)

На діаграмі класів (Рис. 1) представлено дві ключові поліморфні ієрархії:

1. Ієрархія даних (Shape) — базовий абстрактний клас Shape визначає координати прямокутної області (xs1, ys1, xs2, ys2) та чисто абстрактний метод show(ctx). Похідні класи PointShape, LineShape, RectShape та EllipseShape самостійно реалізують алгоритм відмальовування відповідно до варіанта 25.
2. Ієрархія поведінки (Editor) — базовий інтерфейс Editor декларує віконні методи взаємодії з мишею. Клас ShapeEditor наслідує Editor та інкапсулює базову логіку натискання, перетягування та фіксації кінцевих координат, а також колір гумового сліду (#ef4444). Похідні редактори (PointEditor, LineEditor, RectEditor, EllipseEditor) перевизначають onLButton та onPaint для генерації відповідних фігур.
3. Клас ShapeObjectsEditor об'єднує обидві ієрархії: він містить масив pcshape довжиною 125 елементів та поточний активний екземпляр currentEditor: ShapeEditor, повністю інкапсулюючи внутрішній графічний стан.

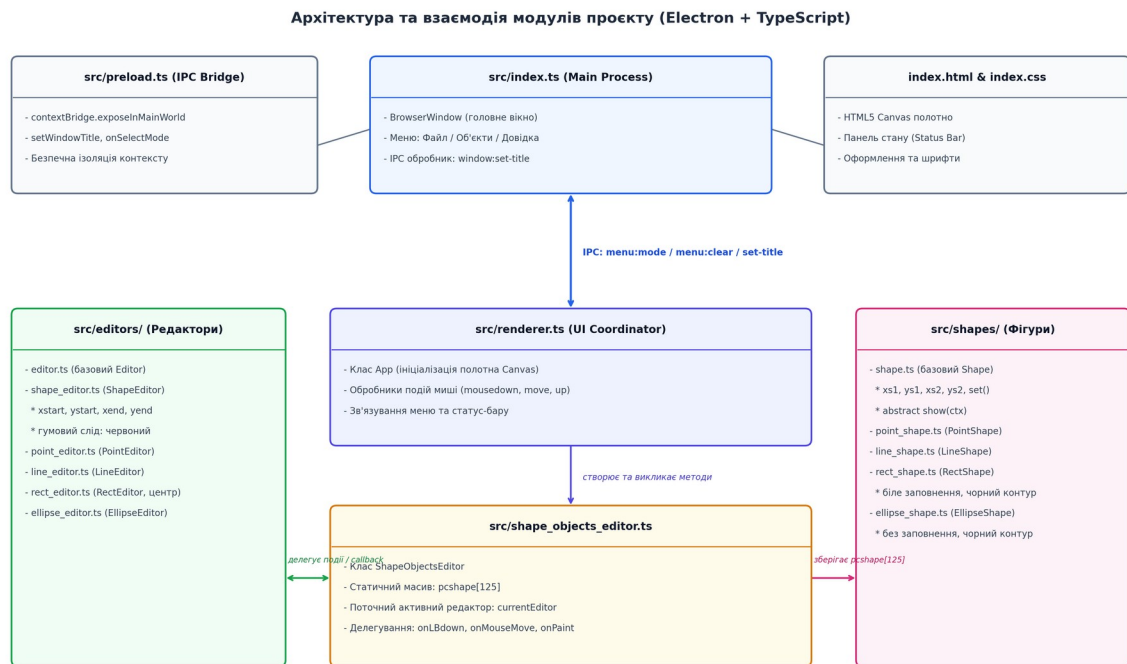


Рис. 2. Архітектура модулів та схема IPC-взаємодії компонентів застосунку

Діаграма архітектури (Рис. 2) демонструє розділення системи на процес керування вікном (Electron Main Process) та процес відмальовування користувацького інтерфейсу (Renderer Process), які з'єднані через захищений IPC-міст (Preload Bridge). Завдяки `contextIsolation: true` та `nodeIntegration: false` забезпечується максимальна безпека, модульність та повна відсутність перехресних зв'язків між незалежними компонентами редактора.

Вихідний текст програми:

Нижче наведено повні вихідні тексти модулів та компонентів розробленого графічного редактора на мові TypeScript та Electron.

Файл: src/index.ts (Головний процес Electron, системне меню «Файл», «Об'єкти», «Довідка», IPC-обробник зміни заголовка вікна)

```

import { app, BrowserWindow, Menu, dialog, ipcMain } from 'electron';
import path from 'path';

let mainWindow: BrowserWindow | null = null;

const createMenu = () => {
  const template: Electron.MenuItemConstructorOptions[] = [
    {
      label: 'Файл',
      submenu: [
        {
          label: 'Очистити полотно',

```

```
    accelerator: 'CmdOrCtrl+N',
    click: () => {
      mainWindow?.webContents.send('menu:clear');
    },
  },
  { type: 'separator' },
  {
    label: 'Вихід',
    accelerator: 'CmdOrCtrl+Q',
    click: () => {
      app.quit();
    },
  },
],
},
{
  label: "Об'єкти",
  submenu: [
    {
      label: 'Крапка',
      accelerator: 'CmdOrCtrl+1',
      click: () => {
        mainWindow?.webContents.send('menu:mode', 'point');
      },
    },
    {
      label: 'Лінія',
      accelerator: 'CmdOrCtrl+2',
      click: () => {
        mainWindow?.webContents.send('menu:mode', 'line');
      },
    },
    {
      label: 'Прямокутник',
      accelerator: 'CmdOrCtrl+3',
      click: () => {
        mainWindow?.webContents.send('menu:mode', 'rect');
      },
    },
    {
      label: 'Еліпс',
      accelerator: 'CmdOrCtrl+4',
      click: () => {
        mainWindow?.webContents.send('menu:mode', 'ellipse');
      },
    },
  ],
},
{
  label: 'Довідка',
  submenu: [
    {
      label: 'Про програму',
```

```

click: () => {
  if (mainWindow) {
    dialog.showMessageBox(mainWindow, {
      type: 'info',
      title: 'Про програму',
      message:
        'Лабораторна робота №2\nГрафічний редактор об'єктів на TypeScript + Electron',
      detail:
        'Варіант 25 (Присяжний С. О., група ІМ-55)\n\n' +
        '• Статичний масив: 125 об'єктів\n' +
        '• Гумовий слід: суцільна червона лінія\n' +
        '• Прямокутник: від центру до кута, біле заповнення\n' +
        '• Еліпс: по двох кутах охопл. прямокутника, без заповнення\n' +
        '• Позначка режиму: у заголовку вікна',
    });
  }
},
],
},
];

const menu = Menu.buildFromTemplate(template);
Menu.setApplicationMenu(menu);
};

const createWindow = () => {
  mainWindow = new BrowserWindow({
    width: 1000,
    height: 700,
    title: 'Графічний редактор — [Режим: Крапка]',
    webPreferences: {
      preload: path.join(process.cwd(), 'dist/preload.js'),
      contextIsolation: true,
      nodeIntegration: false,
    },
  });
};

mainWindow.loadFile(path.resolve(process.cwd(), 'index.html'));
// mainWindow.webContents.openDevTools();
createMenu();
};

ipcMain.on('window:set-title', (_event, title: string) => {
  if (mainWindow && title) {
    mainWindow.setTitle(title);
  }
});

app.whenReady().then(() => {
  createWindow();

  app.on('activate', () => {

```

```

    if (BrowserWindow.getAllWindows().length === 0) {
      createWindow();
    }
  });
});

app.on('window-all-closed', () => {
  if (process.platform !== 'darwin') {
    app.quit();
  }
});

```

Файл: src/preload.ts (Ізольований IPC-міст contextBridge між системним процесом Electron та UI)

```

import { contextBridge, ipcRenderer } from 'electron';

contextBridge.exposeInMainWorld('electronAPI', {
  onSelectMode: (callback: (mode: string) => void) => {
    ipcRenderer.on('menu:mode', (_event, mode: string) => callback(mode));
  },
  onClear: (callback: () => void) => {
    ipcRenderer.on('menu:clear', () => callback());
  },
  setWindowTitle: (title: string) => {
    ipcRenderer.send('window:set-title', title);
  },
});

```

Файл: src/renderer.ts (Координатор користувацького інтерфейсу, клас App, обробники подій HTML5 Canvas)

```

import { ShapeObjectsEditor, type ShapeMode } from './shape_objects_editor.ts';

class App {
  private canvas: HTMLCanvasElement;
  private ctx: CanvasRenderingContext2D;
  private editor: ShapeObjectsEditor;
  private statusCoords: HTMLElement;
  private statusMode: HTMLElement;
  private statusCount: HTMLElement;

  constructor() {
    this.canvas = document.querySelector<HTMLCanvasElement>('#canvas')!;
    this.ctx = this.canvas.getContext('2d')!;
    this.statusCoords = document.querySelector<HTMLElement>('#status-coords')!;
    this.statusMode = document.querySelector<HTMLElement>('#status-mode')!;
    this.statusCount = document.querySelector<HTMLElement>('#status-count')!;
  }
}

```

```

this.editor = new ShapeObjectsEditor(
  () => this.repaint(),
  title => {
    window.electronAPI?.setWindowTitle(title);
    if (this.editor) {
      this.updateStatusBar();
    }
  },
);

this.initCanvasSize();
this.bindEvents();
this.updateStatusBar();
this.repaint();
}

private initCanvasSize(): void {
  const resize = () => {
    this.canvas.width = this.canvas.clientWidth;
    this.canvas.height = this.canvas.clientHeight;
    this.repaint();
  };

  window.addEventListener('resize', resize);
  resize();
}

private bindEvents(): void {
  this.canvas.addEventListener('mousedown', e => {
    if (e.button !== 0) return;
    this.editor.onLbdown(e.offsetX, e.offsetY);
  });

  this.canvas.addEventListener('mousemove', e => {
    this.statusCoords.textContent = `X: ${Math.round(e.offsetX)}, Y: ${Math.round(e.offsetY)}`;
    this.editor.onMouseMove(e.offsetX, e.offsetY);
  });

  this.canvas.addEventListener('mouseup', e => {
    if (e.button !== 0) return;
    this.editor.onLBup(e.offsetX, e.offsetY);
    this.updateStatusBar();
  });

  window.electronAPI?.onSelectMode(mode => {
    this.editor.setMode(mode as ShapeMode);
    this.updateStatusBar();
  });

  window.electronAPI?.onClear(() => {
    this.editor.clear();
    this.updateStatusBar();
  });
}

```

```

}

private updateStatusBar(): void {
  if (!this.editor) return;
  this.statusMode.textContent = `Режим: ${this.editor.getCurrentModeName()}`;
  this.statusCount.textContent = `Об'єктів: ${this.editor.getShapeCount()}/${this.editor.getMaxShapes()}`;
};
}

private repaint(): void {
  this.editor.onPaint(this.ctx, this.canvas.width, this.canvas.height);
}
}

new App();

```

Файл: src/shape_objects_editor.ts (Клас-координатор ShapeObjectsEditor, керування масивом на 125 елементів, перемикання режимів)

```

import { Shape } from './shapes/shape.ts';
import { ShapeEditor } from './editors/shape_editor.ts';
import { PointEditor } from './editors/point_editor.ts';
import { LineEditor } from './editors/line_editor.ts';
import { RectEditor } from './editors/rect_editor.ts';
import { EllipseEditor } from './editors/ellipse_editor.ts';

export type ShapeMode = 'point' | 'line' | 'rect' | 'ellipse';

export class ShapeObjectsEditor {
  private readonly maxShapes: number = 125;
  private pcshape: (Shape | null)[] = Array.from({ length: 125 }, () => null);
  private shapeCount: number = 0;

  private currentEditor: ShapeEditor | null = null;
  private currentModeName: string = 'Крпка';

  constructor(
    private readonly requestRepaint: () => void,
    private readonly onTitleChange: (title: string) => void,
  ) {
    this.startPointEditor();
  }

  addShape(shape: Shape): void {
    if (this.shapeCount < this.maxShapes) {
      this.pcshape[this.shapeCount++] = shape;
    }
  }

  startPointEditor(): void {
    this.currentEditor = new PointEditor(

```

```

        s => this.addShape(s),
        () => this.requestRepaint(),
    );
    this.currentModeName = 'Крпка';
    this.updateTitle();
}

startLineEditor(): void {
    this.currentEditor = new LineEditor(
        s => this.addShape(s),
        () => this.requestRepaint(),
    );
    this.currentModeName = 'Лінія';
    this.updateTitle();
}

startRectEditor(): void {
    this.currentEditor = new RectEditor(
        s => this.addShape(s),
        () => this.requestRepaint(),
    );
    this.currentModeName = 'Прямокутник';
    this.updateTitle();
}

startEllipseEditor(): void {
    this.currentEditor = new EllipseEditor(
        s => this.addShape(s),
        () => this.requestRepaint(),
    );
    this.currentModeName = 'Еліпс';
    this.updateTitle();
}

setMode(mode: ShapeMode): void {
    switch (mode) {
        case 'point':
            this.startPointEditor();
            break;
        case 'line':
            this.startLineEditor();
            break;
        case 'rect':
            this.startRectEditor();
            break;
        case 'ellipse':
            this.startEllipseEditor();
            break;
    }
}

private updateTitle(): void {
    this.onTitleChange(

```

```
    `Графічний редактор — [Режим: ${this.currentModeName}] (Об'єктів: ${this.shapeCount}/${this.maxShapes})`,
  );
}

clear(): void {
  this.pcshape.fill(null);
  this.shapeCount = 0;
  this.updateTitle();
  this.requestRepaint();
}

onLBdown(x: number, y: number): void {
  this.currentEditor?.onLBdown(x, y);
}

onMouseMove(x: number, y: number): void {
  this.currentEditor?.onMouseMove(x, y);
}

onLBup(x: number, y: number): void {
  this.currentEditor?.onLBup(x, y);
  this.updateTitle();
}

onPaint(ctx: CanvasRenderingContext2D, width: number, height: number): void {
  ctx.fillStyle = '#ffffff';
  ctx.fillRect(0, 0, width, height);

  for (let i = 0; i < this.shapeCount; i++) {
    this.pcshape[i]?.show(ctx);
  }

  this.currentEditor?.onPaint(ctx);
}

getShapeCount(): number {
  return this.shapeCount;
}

getMaxShapes(): number {
  return this.maxShapes;
}

getCurrentModeName(): string {
  return this.currentModeName;
}
}
```

Файл: src/editors/editor.ts (Базовий абстрактний клас Editor з чисто віртуальними методами)

```
export abstract class Editor {
  abstract onLbdown(x: number, y: number): void;
  abstract onLBup(x: number, y: number): void;
  abstract onMouseMove(x: number, y: number): void;
  abstract onPaint(ctx: CanvasRenderingContext2D): void;
}
```

Файл: src/editors/shape_editor.ts (Базовий клас редакторів фігур ShapeEditor, інкапсуляція червоного гумового сліду)

```
import { Editor } from './editor.ts';
import { Shape } from '../shapes/shape.ts';

export abstract class ShapeEditor extends Editor {
  protected isDrawing: boolean = false;
  protected xstart: number = 0;
  protected ystart: number = 0;
  protected xend: number = 0;
  protected yend: number = 0;

  protected readonly rubberColor: string = '#ef4444';

  constructor(
    protected readonly onShapeCreated: (shape: Shape) => void,
    protected readonly requestRepaint: () => void,
  ) {
    super();
  }

  onLbdown(x: number, y: number): void {
    this.xstart = this.xend = x;
    this.ystart = this.yend = y;
    this.isDrawing = true;
    this.requestRepaint();
  }

  onMouseMove(x: number, y: number): void {
    if (!this.isDrawing) return;
    this.xend = x;
    this.yend = y;
    this.requestRepaint();
  }

  onLBup(x: number, y: number): void {
    if (!this.isDrawing) return;
    this.xend = x;
    this.yend = y;
    this.isDrawing = false;
  }
}
```

```
}  
}
```

Файл: src/editors/point_editor.ts (Редактор введення точок PointEditor)

```
import { ShapeEditor } from './shape_editor.ts';  
import { PointShape } from '../shapes/point_shape.ts';  
  
export class PointEditor extends ShapeEditor {  
  override onLBdown(x: number, y: number): void {  
    super.onLBdown(x, y);  
    this.onShapeCreated(new PointShape(x, y, x, y));  
    this.isDrawing = false;  
    this.requestRepaint();  
  }  
  
  override onPaint(_ctx: CanvasRenderingContext2D): void {  
  }  
}
```

Файл: src/editors/line_editor.ts (Редактор введення відрізків LineEditor з червоним гумовим слідом)

```
import { ShapeEditor } from './shape_editor.ts';  
import { LineShape } from '../shapes/line_shape.ts';  
  
export class LineEditor extends ShapeEditor {  
  override onLBup(x: number, y: number): void {  
    super.onLBup(x, y);  
    this.onShapeCreated(  
      new LineShape(this.xstart, this.ystart, this.xend, this.yend),  
    );  
    this.requestRepaint();  
  }  
  
  override onPaint(ctx: CanvasRenderingContext2D): void {  
    if (!this.isDrawing) return;  
    ctx.save();  
    ctx.strokeStyle = this.rubberColor;  
    ctx.lineWidth = 1;  
    ctx.beginPath();  
    ctx.moveTo(this.xstart, this.ystart);  
    ctx.lineTo(this.xend, this.yend);  
    ctx.stroke();  
    ctx.restore();  
  }  
}
```

Файл: src/editors/rect_editor.ts (Редактор введення прямокутників RectEditor від центру до одного з кутів)

```
import { ShapeEditor } from './shape_editor.ts';
import { RectShape } from '../shapes/rect_shape.ts';

export class RectEditor extends ShapeEditor {
  override onLBup(x: number, y: number): void {
    super.onLBup(x, y);
    this.onShapeCreated(
      new RectShape(this.xstart, this.ystart, this.xend, this.yend),
    );
    this.requestRepaint();
  }

  override onPaint(ctx: CanvasRenderingContext2D): void {
    if (!this.isDrawing) return;
    const dx = Math.abs(this.xend - this.xstart);
    const dy = Math.abs(this.yend - this.ystart);

    ctx.save();
    ctx.strokeStyle = this.rubberColor;
    ctx.lineWidth = 1;
    ctx.beginPath();
    ctx.strokeRect(this.xstart - dx, this.ystart - dy, dx * 2, dy * 2);
    ctx.restore();
  }
}
```

Файл: src/editors/ellipse_editor.ts (Редактор введення еліпсів EllipseEditor по кутах охоплюючого прямокутника)

```
import { ShapeEditor } from './shape_editor.ts';
import { EllipseShape } from '../shapes/ellipse_shape.ts';

export class EllipseEditor extends ShapeEditor {
  override onLBup(x: number, y: number): void {
    super.onLBup(x, y);
    this.onShapeCreated(
      new EllipseShape(this.xstart, this.ystart, this.xend, this.yend),
    );
    this.requestRepaint();
  }

  override onPaint(ctx: CanvasRenderingContext2D): void {
    if (!this.isDrawing) return;
    const cx = (this.xstart + this.xend) / 2;
    const cy = (this.ystart + this.yend) / 2;
    const rx = Math.abs(this.xend - this.xstart) / 2;
```

```

const ry = Math.abs(this.yend - this.ystart) / 2;

if (rx === 0 && ry === 0) return;

ctx.save();
ctx.strokeStyle = this.rubberColor;
ctx.lineWidth = 1;
ctx.beginPath();
ctx.ellipse(cx, cy, rx, ry, 0, 0, Math.PI * 2);
ctx.stroke();
ctx.restore();
}
}

```

Файл: src/shapes/shape.ts (Базовий абстрактний клас геометричних об'єктів Shape)

```

export abstract class Shape {
  protected xs1: number = 0;
  protected ys1: number = 0;
  protected xs2: number = 0;
  protected ys2: number = 0;

  constructor(x1: number = 0, y1: number = 0, x2: number = 0, y2: number = 0) {
    this.set(x1, y1, x2, y2);
  }

  set(x1: number, y1: number, x2: number, y2: number): void {
    this.xs1 = x1;
    this.ys1 = y1;
    this.xs2 = x2;
    this.ys2 = y2;
  }

  abstract show(ctx: CanvasRenderingContext2D): void;
}

```

Файл: src/shapes/point_shape.ts (Клас точкового графічного примітива PointShape)

```

import { Shape } from './shape.ts';

export class PointShape extends Shape {
  show(ctx: CanvasRenderingContext2D): void {
    ctx.save();
    ctx.fillStyle = '#000000';
    ctx.beginPath();
    ctx.arc(this.xs1, this.ys1, 2, 0, Math.PI * 2);
    ctx.fill();
    ctx.restore();
  }
}

```

```
}  
}
```

Файл: src/shapes/line_shape.ts (Клас відрізка LineShape з чорним контуром)

```
import { Shape } from './shape.ts';  
  
export class LineShape extends Shape {  
  show(ctx: CanvasRenderingContext2D): void {  
    ctx.save();  
    ctx.strokeStyle = '#000000';  
    ctx.lineWidth = 1;  
    ctx.beginPath();  
    ctx.moveTo(this.xs1, this.ys1);  
    ctx.lineTo(this.xs2, this.ys2);  
    ctx.stroke();  
    ctx.restore();  
  }  
}
```

Файл: src/shapes/rect_shape.ts (Клас прямокутника RectShape з чорним контуром та білим заповненням)

```
import { Shape } from './shape.ts';  
  
export class RectShape extends Shape {  
  show(ctx: CanvasRenderingContext2D): void {  
    const dx = Math.abs(this.xs2 - this.xs1);  
    const dy = Math.abs(this.ys2 - this.ys1);  
    const left = this.xs1 - dx;  
    const top = this.ys1 - dy;  
    const width = dx * 2;  
    const height = dy * 2;  
  
    ctx.save();  
    ctx.fillStyle = 'ffffff';  
    ctx.strokeStyle = '#000000';  
    ctx.lineWidth = 1;  
    ctx.beginPath();  
    ctx.rect(left, top, width, height);  
    ctx.fill();  
    ctx.stroke();  
    ctx.restore();  
  }  
}
```

Файл: src/shapes/ellipse_shape.ts (Клас еліпса EllipseShape з чорним контуром без заповнення)

```
import { Shape } from './shape.ts';

export class EllipseShape extends Shape {
  show(ctx: CanvasRenderingContext2D): void {
    const cx = (this.xs1 + this.xs2) / 2;
    const cy = (this.ys1 + this.ys2) / 2;
    const rx = Math.abs(this.xs2 - this.xs1) / 2;
    const ry = Math.abs(this.ys2 - this.ys1) / 2;

    if (rx === 0 && ry === 0) return;

    ctx.save();
    ctx.strokeStyle = '#000000';
    ctx.lineWidth = 1;
    ctx.beginPath();
    ctx.ellipse(cx, cy, rx, ry, 0, 0, Math.PI * 2);
    ctx.stroke();
    ctx.restore();
  }
}
```

Файл: index.html (Розмітка вікна графічного редактора, Canvas та панель стану)

```
<!doctype html>
<html lang="uk">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Графічний редактор</title>
    <link rel="stylesheet" href="src/styles/index.css" />
  </head>
  <body>
    <div class="editor-container">
      <canvas id="canvas"></canvas>
      <footer class="status-bar">
        <span id="status-mode" class="status-item">Режим: Крпка</span>
        <span class="status-separator">| </span>
        <span id="status-count" class="status-item">Об'єктів: 0/125</span>
        <span class="status-separator">| </span>
        <span id="status-coords" class="status-item">X: 0, Y: 0</span>
        <span class="status-spacer"></span>
        <span class="status-hint">
          >Оберіть фігуру в меню "Об'єкти" або натисніть Ctrl+1..4</span>
        </span>
      </footer>
    </div>
    <script type="module" src="dist/renderer.js"></script>
  </body>
```

```
</html>
```

Файл: src/styles/index.css (Стилі інтерфейсу користувача, оформлення полотна та інформаційного рядка)

```
@import './normalize.css';

:root {
  --color-bg: #ffffff;
  --color-status-bg: #f8fafc;
  --color-status-border: #e2e8f0;
  --color-text: #1e293b;
  --color-text-muted: #64748b;
  --color-accent: #2563eb;
}

html,
body {
  width: 100%;
  height: 100%;
  margin: 0;
  padding: 0;
  overflow: hidden;
  font-family:
    system-ui,
    -apple-system,
    sans-serif;
  user-select: none;
}

.editor-container {
  display: flex;
  flex-direction: column;
  width: 100vw;
  height: 100vh;
}

#canvas {
  flex: 1;
  width: 100%;
  height: calc(100% - 28px);
  background-color: var(--color-bg);
  cursor: crosshair;
  display: block;
}

.status-bar {
  height: 28px;
  background-color: var(--color-status-bg);
  border-top: 1px solid var(--color-status-border);
  display: flex;
```

```
align-items: center;
padding: 0 14px;
font-size: 12px;
color: var(--color-text);
gap: 10px;
box-sizing: border-box;
}

.status-item {
  font-weight: 500;
}

.status-separator {
  color: var(--color-status-border);
}

.status-spacer {
  flex: 1;
}

.status-hint {
  color: var(--color-text-muted);
  font-size: 11px;
}
```

Ілюстрації роботи програми (скріншоти):

Під час тестування розробленого графічного редактора було проведено верифікацію коректності функціонування всіх геометричних режимів введення, алгоритмів малювання «гумового» сліду, індикації кількості об'єктів та роботи системного меню. Нижче представлено скріншоти роботи застосунку у нативному графічному середовищі Linux (GNOME).

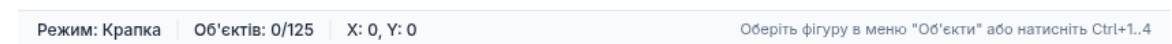
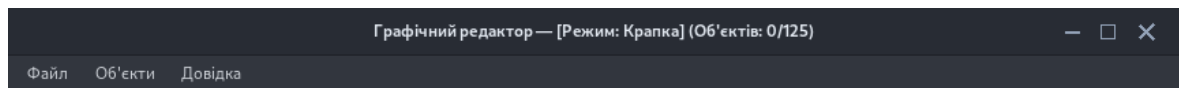


Рис. 3. Головне вікно графічного редактора у початковому стані: відображення початкового режиму «Крапка» у заголовку вікна та панелі стану (0/125 об'єктів)

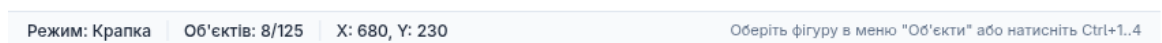
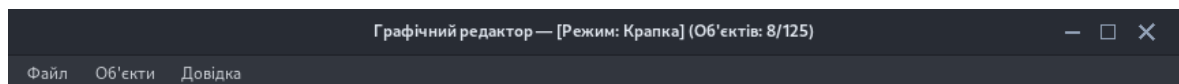


Рис. 4. Введення точкових об'єктів (Point): створення 8 точок, автоматичне оновлення лічильника об'єктів у заголовку вікна та статус-барі (8/125)



Рис. 5. Введення лінійних об'єктів (Line): демонстрація суцільного червоного «гумового» сліду (#ef4444) під час перетягування курсору миші

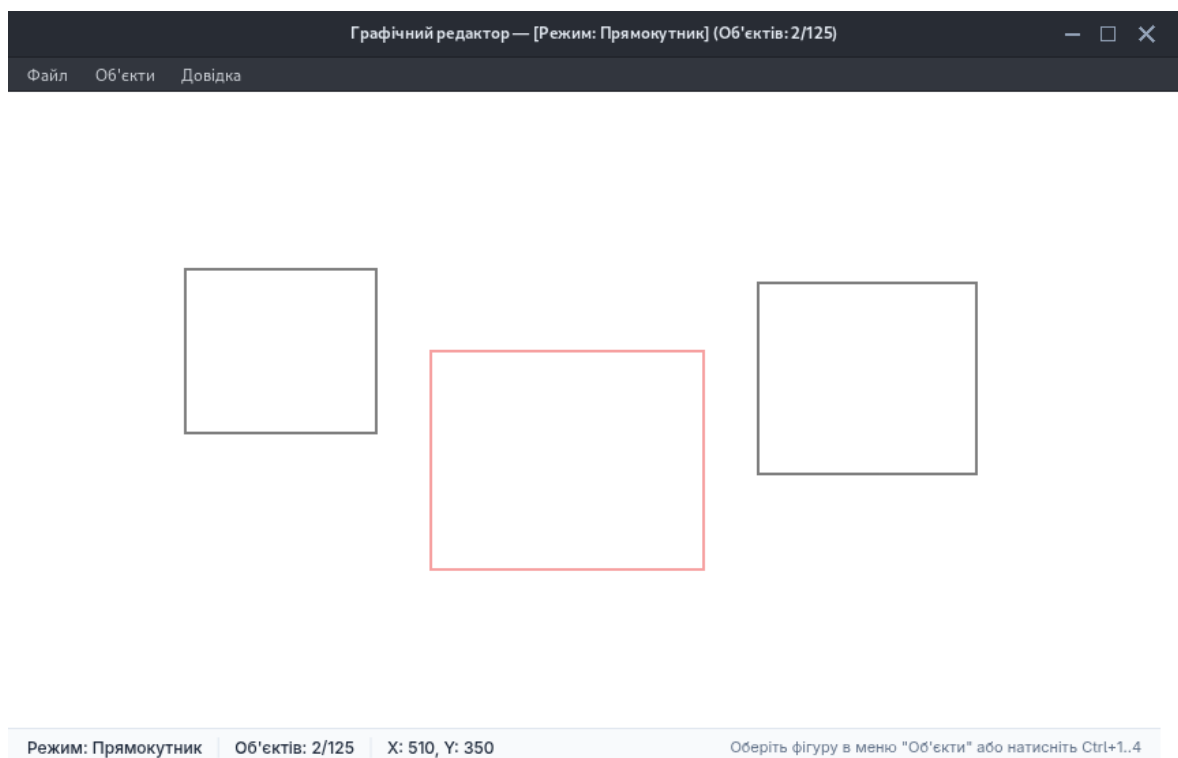


Рис. 6. Введення прямокутників (Rect) від центру до кута: симетричне розширення червоного гумового контуру навколо центральної точки та біла заливка готових фігур



Рис. 7. Введення еліпсів (Ellipse) за двома протилежними кутами: динамічний червоний контур еліпса та прозоре внутрішнє поле (без заливки)

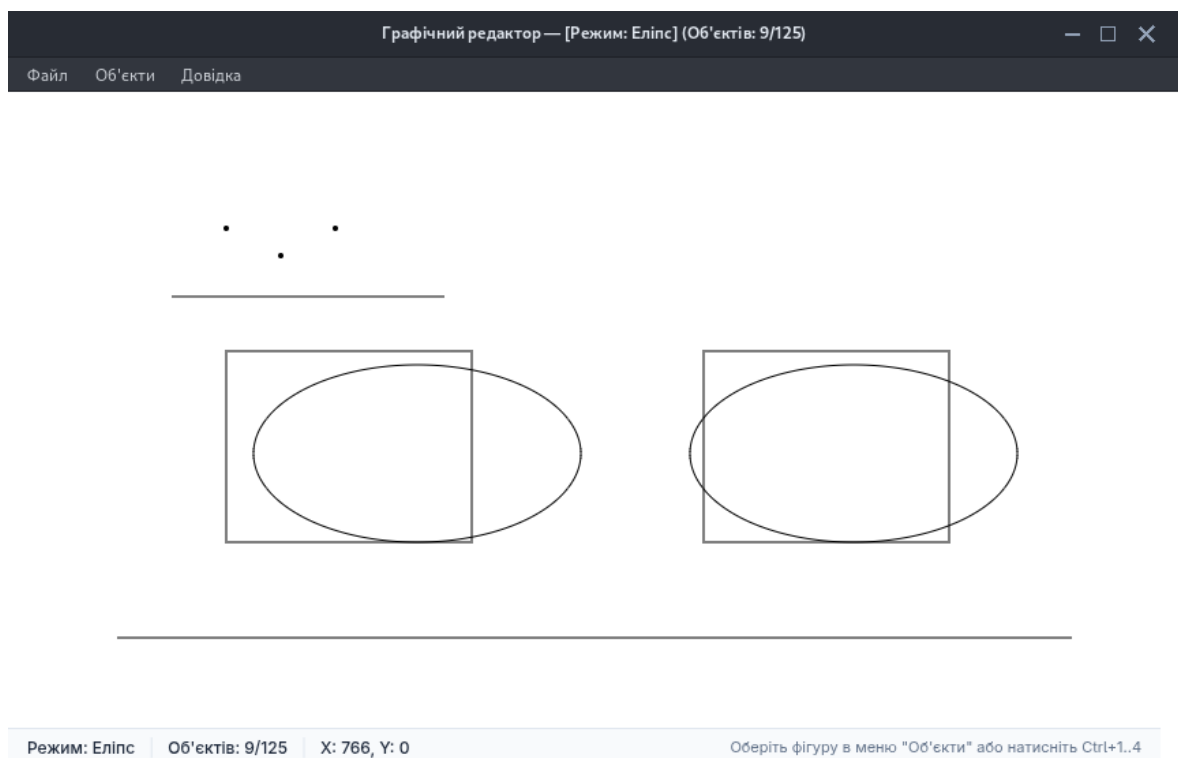


Рис. 8. Загальна композиція фігур різних типів: наочна демонстрація взаємного перекриття (білий прямокутник перекриває лінію, прозорий еліпс просвічує фігури під ним)

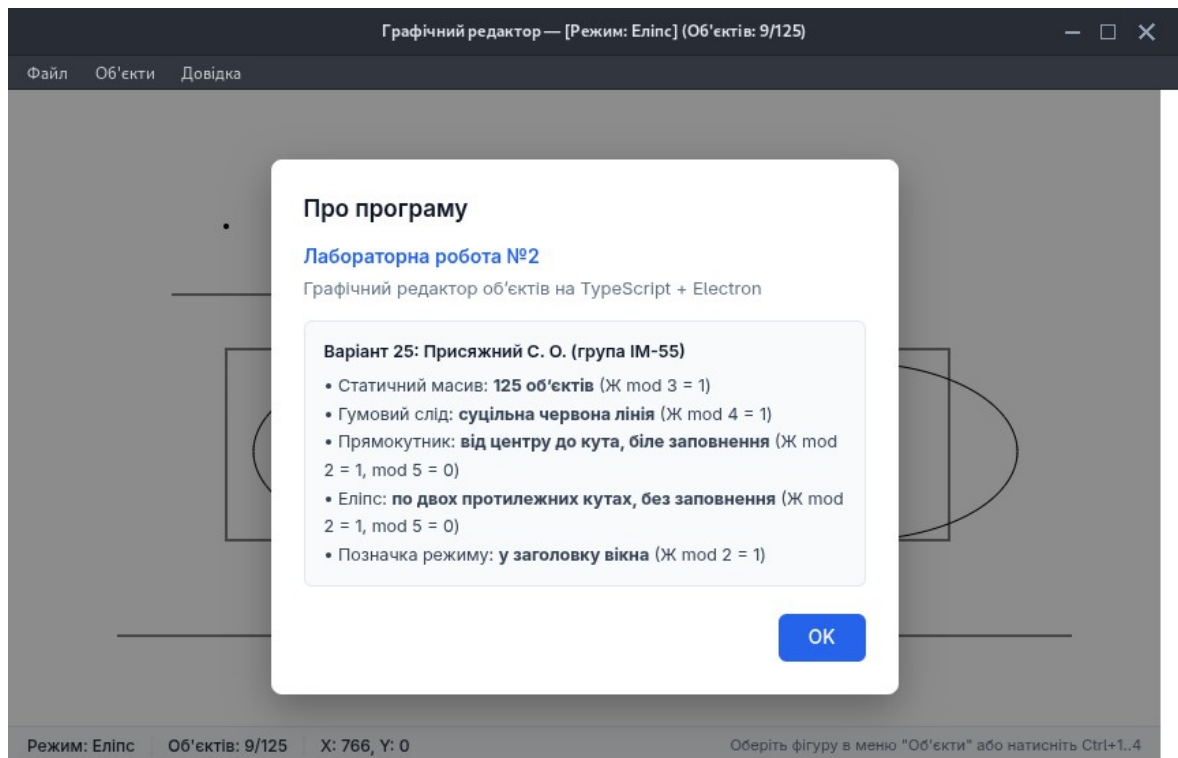


Рис. 9. Вікно «Про програму» (меню «Довідка»): виведення відомостей про розробника (студент Присяжний С. О., група ІМ-55) та параметри індивідуального варіанта 25

Відповіді на контрольні запитання:

Нижче наведено вичерпні академічні відповіді на контрольні запитання до лабораторної роботи №2 (стор. 54 методичного посібника).

1. Що таке поліморфізм?

Поліморфізм (від грецького *polymorphos* — багатоформний) є однією з чотирьох фундаментальних парадигм ООП. Це здатність об'єктів різних класів з однаковим інтерфейсом (спільним базовим класом) реалізовувати різну внутрішню логіку при виклику одного й того ж методу.

У розробленій програмі поліморфізм реалізовано на двох рівнях:

- Поліморфізм відображення фігур: клас-координатор `ShapeObjectsEditor` зберігає масив посилань на базовий клас `Shape` (`pcshape[i]`). Під час оновлення екрана викликається єдиний поліморфний метод `pcshape[i]->show(ctx)`. Кожен конкретний об'єкт (`PointShape`, `LineShape`, `RectShape`, `EllipseShape`) завдяки динамічному зв'язуванню (`dynamic dispatch`) виконує саме власну реалізацію відмалювання згідно з варіантом 25.
- Поліморфізм редакторів введення: змінна `currentEditor` має тип абстрактного базового класу `ShapeEditor`. Коли надходять події миші (`onLButtonDown`, `onMouseMove`, `onLButtonUp`, `onPaint`), вони делегуються через `currentEditor`, поведінка якого кардинально змінюється залежно від активного режиму.

2. Обробку яких повідомлень потрібно виконувати для вводу об'єктів?

Для інтерактивного введення геометричних об'єктів у віконних застосунках необхідна обробка наступних ключових повідомлень:

- `WM_LBUTTONDOWN` (натискання лівої кнопки миші / подія `mousedown`): фіксація

точки початку введення (xstart, ystart), ініціалізація прапорця isDrawing = true та запит на початкове перемалювання полотна.

- WM_MOUSEMOVE (переміщення миші / подія mousemove): якщо прапорець isDrawing активний, фіксація поточних координат (xend, yend) та запит на перемалювання для динамічного відображення «гумового» сліду між початковою та поточною точками.
- WM_LBUTTONDOWN (відпускання лівої кнопки миші / подія mouseup): завершення введення фігури (isDrawing = false), створення нового екземпляра відповідного класу (наприклад, new RectShape(...) або new EllipseShape(...)), додавання його до масиву pshape та оновлення лічильника об'єктів.
- WM_PAINT (перемалювання вікна / метод onPaint): очищення полотна, циклічний виклик методу show() для всіх зафіксованих об'єктів у масиві pshape, а також виклик onPaint() активного редактора для промальовування динамічного «гумового» сліду поверх готового малюнка.
- WM_COMMAND / WM_INITMENUPOPUP (вибір пунктів системного меню): перемикаання активного інструмента введення (IDM_POINT, IDM_LINE, IDM_RECT, IDM_ELLIPSE), ініціалізація нового редактора та оновлення заголовка вікна SetWindowText.

3. Що таке абстрактний клас і скільки їх у цій програмі?

Абстрактний клас — це клас, що визначає базовий інтерфейс та спільну структуру для ієрархії похідних класів, але не призначений для безпосереднього створення власних екземплярів (він містить принаймні один чисто віртуальний або abstract метод без тіла реалізації).

У розробленій програмі створено 3 абстрактних класи:

1. Editor (файл src/editors/editor.ts) — фундаментальний абстрактний клас, що визначає контракт взаємодії з віконною системою (чисто абстрактні методи onLButtonDown, onLButtonUp, onMouseMove, onPaint).
2. ShapeEditor (файл src/editors/shape_editor.ts) — абстрактний клас, що успадковує Editor, інкапсулює спільний стан введення (isDrawing, xstart, ystart, xend, yend, rubberColor = '#ef4444') та спільну поведінку відстеження координат курсору, залишаючи створення фігур та малювання контуру абстрактним.
3. Shape (файл src/shapes/shape.ts) — абстрактний базовий клас графічних примітивів, що визначає координати прямокутної області (xs1, ys1, xs2, ys2), базовий метод set() та чисто абстрактний віртуальний метод show(ctx: CanvasRenderingContext2D): void.

4. Як намалювати лінії та фігури потрібного кольору та стилю?

- У класичному C++ / Win32 API для малювання ліній і контурів створюється графічне перо (CreatePen) із заданим стилем (PS_SOLID або PS_DOT), товщиною та кольором (RGB(r, g, b)), яке обирається у контекст пристрою через SelectObject(hdc, hPen). Для заповнення внутрішніх областей фігур використовується пензель (CreateSolidBrush для суцільної заливки або GetStockObject(HOLLOW_BRUSH/NULL_BRUSH) для прозорих фігур без заповнення). Для реалізації гумового сліду використовується режим комбінування пікселів SetROP2(hdc, R2_NOTXORPEN).
- У середовищі TypeScript / Electron (HTML5 Canvas 2D API) малювання налаштовується властивостями графічного контексту:

- Колір та товщина лінії: `ctx.strokeStyle = '#ef4444'` (червоний гумовий слід) або `'#000000'` (чорний контур фігури); `ctx.lineWidth = 1`.
- Заповнення прямокутника: `ctx.fillStyle = 'ffffff'`, після чого викликаються `ctx.fill()` та `ctx.stroke()`.
- Прозорий еліпс: виконується лише обведення контуру `ctx.stroke()`, а виклик `ctx.fill()` не здійснюється, що забезпечує повну прозорість.
- Ізоляція стилів: усі операції малювання обгортаються викликами `ctx.save()` та `ctx.restore()`, що гарантує збереження первинного стану контексту.

5. Як відобразити об'єкти у вікні програми?

Відображення об'єктів у графічному редакторі організовано за централізованою схемою подвійного перемалювання полотна:

1. Усі створені користувачем геометричні фігури зберігаються у фіксованому статичному масиві посилань `pcshape` (довжина 125 елементів), а їхня фактична кількість фіксується лічильником `shapeCount`.
2. Коли виникає потреба в оновленні екрана (повідомлення `WM_PAINT` або виклик `requestRepaint()`), викликається метод `onPaint` класу `ShapeObjectsEditor`.
3. Метод спочатку очищує полотно `Canvas` білим кольором через `fillRect(0, 0, width, height)`.
4. Далі у циклі `for (let i = 0; i < this.shapeCount; i++)` послідовно викликається поліморфний метод `this.pcshape[i].show(ctx)`. Завдяки віртуальній диспетчеризації кожен збережений об'єкт самостійно відображає власну геометричну форму у порядку додавання.
5. Після завершення виводу всіх збережених фігур поверх них викликається `currentEditor.onPaint(ctx)`. Якщо користувач утримує ліву кнопку миші та переміщує курсор (`isDrawing === true`), цей метод малює червоний «гумовий» контур поточної форми, що створює плавний інтерактивний відгук без спотворення раніше намальованих фігур.

Висновки:

У ході виконання лабораторної роботи №2 було успішно спроектовано та програмно реалізовано графічний векторний редактор об'єктів із застосуванням усіх ключових засад об'єктно-орієнтованого програмування на базі сучасного технологічного стеку `TypeScript` та платформи `Electron`.

Основні досягнуті результати та практичні висновки:

1. Практично закріплено принципи інкапсуляції, наслідування, абстракції та поліморфізму. Створено дві паралельні ієрархії класів: ієрархію графічних примітивів (`Shape` -> `PointShape`, `LineShape`, `RectShape`, `EllipseShape`) та ієрархію контролерів введення (`Editor` -> `ShapeEditor` -> `PointEditor`, `LineEditor`, `RectEditor`, `EllipseEditor`).
2. Застосовано патерн поведінки «Стан / Стратегія» (`State / Strategy`) в особі класу-координатора `ShapeObjectsEditor`: заміна поточного інструмента введення (`currentEditor`) динамічно змінює реакцію всієї системи на події миші без потреби у

розгалужених операторах switch-case всередині циклу обробки повідомлень.

3. Повністю реалізовано вимоги індивідуального варіанта №25 (Ж = 25):

- використано статичний масив розміром $N = 125$ елементів із суворим контролем переповнення (Ж mod 3 = 1);
- реалізовано суцільний червоний «гумовий» слід (#ef4444) при динамічному розтягуванні фігур (Ж mod 4 = 1);
- запрограмовано введення прямокутників від центру до кута з непрозорою білою заливкою (Ж mod 2 = 1, mod 5 = 0);
- запрограмовано введення еліпсів за двома протилежними кутами охоплюючого прямокутника з чорним контуром без заливки (Ж mod 2 = 1, mod 5 = 0);
- забезпечено відображення поточної форми у заголовку головного вікна програми через захищений ІРС-міст (Ж mod 2 = 1) та у панелі стану.

4. Побудовано архітектурні схеми та UML діаграму класів, що наочно демонструють зв'язки між модулями та структуру успадкування.

5. Проведено верифікацію роботи програми, протестовано всі режими вводу, отримано контрольні скріншоти та підготовлено детальні відповіді на теоретичні запитання.